Mini Moodle API: Modern LMS Platform
Kosmurat Samat, Bauyrzhan Nurzhanov

Mini Moodle is a comprehensive Learning Management System (LMS) developed in Go. It provides all the necessary tools for managing courses, assignments, grades, and interaction between students and instructors.

Technology Stack Overview

Programming Language

Go 1.25

Framework

Gin

Database

PostgreSQL

DB Driver

PGX

Authentication

JWT

Documentation

Swagger/OpenAPI

Core Functionality

User Management

- Registration and Authentication (JWT)
- Role Management (student, instructor, administrator, etc.)
- Token Refresh Mechanism
- User Profiles

Course Management

- Create, Edit, Delete Courses
- Course Categorization
- Student Enrollment Management
- Progress Tracking

Assignments and Grades

- Create Assignments with Deadlines
- Submission of Student Work
- Grading System
- Feedback from Instructors
- Grade History

Tests and Quizzes

- Multiple Choice and Open-Ended Questions
- Automatic Answer Checking
- Results Statistics
- Time Limits

Attendance

- Student Attendance Tracking
- Attendance Sessions
- Absence Reports

Communication and Notifications

- Chat System between Students and Instructors
- Real-time Notifications
- Course Announcements
- WebSocket Support

Administration and Appeals

Administration

- Admin Panel
- teacher
- Manager and manager categories
- student

Appeals

- Grade Appeal System
- Administrator Review Process
- Status History

Mini Moodle offers a full suite of features for effective platform management. From detailed control over users and courses to a sophisticated appeal review system ensuring fairness and transparency.

The administration and appeals modules provide robust support for maintaining order and efficiency across the entire educational ecosystem.

API Modules and Project Structure

API Modules (22 handlers)

auth, user, course, enrollment, assignment, submission, grade, quiz, attendance, chat, notification, announcement, appeal, dashboard, group, teacher, student, upload, admin, manager, categorymanager, coursecategory

Security

- JWT Token Authentication
- Bearer Token Scheme
- Role-Based Access Control (RBAC)
- CORS Middleware
- Password Hashing
- SQL Injection Protection
- Request Validation
- Timeout Protection

Project Structure

01

cmd/

API server entry points and database migration tool.

02

internal/

Application initialization, HTTP handlers, business models, data access layer, business logic, and common utilities.

03

docs/

Swagger documentation.

04

migrations/

SQL migrations (11 versions).

Quick Start: Local Installation

Commands to Run

```
$ go mod download
```

```
$ swag init -g cmd/api/main.go -o docs
```

```
$ go run cmd/migrate/main.go up
```

```
$ go run cmd/api/main.go
```

Using Docker

```
$ docker-compose up -d
```

API Access

http://localhost:8080

Swagger UI Documentation

http://localhost:8080/swagger/index.html

Typical API Workflow

1. User Registration

`POST /auth/register`

Create a new user account with specified username, email, password, and role.

2. User Login

`POST /auth/login`

Obtain a JWT token for authentication in subsequent requests.

3. Token Usage

Add the header `Authorization: Bearer YOUR_TOKEN` to all protected requests.

4. Course Creation (for instructor)

`POST /courses`

Create a new course with a name, description, code, and number of credits.

5. Student Enrollment

`POST /enrollments`

Enroll a student in a specific course.

6. Assignment Creation

`POST /assignments`

Add a new assignment for a course with a title, description, and due date.

7. Submission (for student)

`POST /submissions`

Submit an assignment solution, including content and file identifiers.

8. Grading (for instructor)

`POST /grades`

Assign a grade and add feedback to a student's submission.

Testing with Swagger UI

How to Use

- Open http://localhost:8080/swagger/index.html
- Click "Try it out" on any endpoint.
- Fill in the request parameters.
- Click "Execute" to send the request.
- View the server response.

Authorization

- Click the "Authorize" button (top right corner).
- Enter: `Bearer YOUR_TOKEN`.
- Click "Authorize".
- All subsequent requests will use the provided token.

Application Lifecycle and Background Tasks

Initialization

- Configuration Loading
- DB Connection
- Dependency Creation
- HTTP Router Setup

Operation

- HTTP Server (goroutine)
- Background Workers (notification handler, health check, error handler)
- Signal Handler (SIGINT, SIGTERM)

Shutdown

- 10-second Timeout
- Closing Active Connections
- Stopping Background Tasks
- Closing Database
- Graceful Exit

Background Workers

- **Notification Handler:** processes notifications every 5 seconds.
- **Health Check:** checks database every 30 seconds.
- **Error Handler:** handles application errors.

Ready to Use!

HTTP Modules

All 22 API modules are fully functional.

Configured

Database is configured and ready for use.

Documentation

Complete Swagger API documentation for easy integration.

Get Started Now:

```
$ go run cmd/api/main.go
```

Visit **http://localhost:8080/swagger/index.html**